

项目需求分析及相关技术调研文档

CodeMate: AI 辅助代码评审与测试用例生成系统

课程主题: ChatGPT / Bing Copilot AI 辅助软件开发

课程名称: 软件实现技术_____

小组成员: 罗嘉滨, 张嘉豪, 徐昕博, 王正阳

指导教师: 蒋严冰_____

提交日期: 2026 年 5 月 9 日

摘要

随着 ChatGPT、GitHub Copilot、Bing Copilot 等生成式 AI 工具进入软件开发流程，开发者可以在需求理解、代码生成、代码审查、测试生成和文档编写等环节获得辅助。本项目拟设计并实现一套面向学生团队和中小型开发团队的 AI 辅助代码评审与测试用例生成系统 CodeMate。系统允许用户上传代码文件或 Git diff，通过规则预处理、静态分析和大语言模型推理生成结构化审查结果，包括问题类型、严重程度、代码位置、问题说明、修改建议和测试用例建议。系统强调“AI 辅助、人工确认”的开发原则，避免将 AI 输出直接作为最终结论。本文档从项目背景、用户角色、业务流程、功能需求、非功能需求、技术架构、数据设计、AI 提示词与结构化输出、安全风险、测试评估和可行性等方面进行分析，并结合 GitHub Copilot、OpenAI Structured Outputs、Microsoft Copilot Studio、OWASP LLM Top 10、NIST AI RMF 与 SSDF 等公开资料进行技术调研。

关键词：AI 辅助软件开发；代码评审；测试用例生成；ChatGPT；Bing Copilot；GitHub Copilot；大语言模型；结构化输出；软件工程

目录

- 摘要 2
- 1 项目概述 5
 - 1.1 项目背景 5
 - 1.2 项目名称与定位 5
 - 1.3 项目目标 5
- 2 现状与问题分析 5
 - 2.1 当前软件开发中的痛点 5
 - 2.2 项目机会 6
- 3 需求分析 6
 - 3.1 相关方与用户角色 6
 - 3.2 业务流程 6
 - 3.3 功能需求 6
 - 3.4 非功能需求 7
 - 3.5 典型用例 7
 - 3.6 需求优先级 7
- 4 系统总体设计 8
 - 4.1 总体架构 8
 - 4.2 推荐技术栈 8
 - 4.3 核心模块设计 8
 - 4.4 数据库设计 9
 - 4.5 AI 输出结构设计 9
- 5 关键技术调研 10
 - 5.1 ChatGPT / Copilot 辅助软件开发能力 10
 - 5.2 结构化输出技术 10
 - 5.3 静态分析与 LLM 结合 10
 - 5.4 企业知识源与 RAG 可扩展性 11
 - 5.5 安全与风险管理技术 11
- 6 安全、隐私与合规分析 11
 - 6.1 安全策略 11
- 7 测试与评价方案 12

7.1 功能测试	12
7.2 AI 效果评价指标	12
7.3 测试数据设计	12
8 项目实施计划与可行性分析	12
8.1 里程碑计划	12
8.2 可行性分析	13
8.3 风险与应对	13
9 总结	13
附录 A: AI 输出 JSON 示例	14
附录 B: 需求追踪矩阵	14
参考文献	15

1 项目概述

1.1 项目背景

软件开发活动通常包括需求分析、设计、编码、测试、代码评审、部署和维护等环节。对于学生团队和小型开发团队而言，代码审查和测试用例编写经常受到时间、经验和人力资源限制，导致潜在缺陷无法及时发现，测试覆盖不足，代码风格不统一。

以 GitHub Copilot 为代表的 AI 编程助手已经支持代码评审、测试生成、PR 摘要、提交信息生成和云端代理式开发等能力。GitHub 官方文档说明，Copilot code review 可审查多种语言代码，并从多个角度识别问题和建议修复；但官方同时强调，Copilot 并不能保证发现所有问题，开发者需要仔细验证反馈并辅以人工评审 [1]。

因此，本项目不将 AI 设计为完全替代开发者的系统，而是定位为“预审查、预测试、辅助学习”的工具：系统先自动发现可疑问题并生成测试建议，再由开发者或评审者确认、修改或忽略。

1.2 项目名称与定位

项目项	内容
项目名称	CodeMate: AI 辅助代码评审与测试用例生成系统
项目类型	AI 辅助软件开发工具 / 软件工程实践平台
目标用户	学生开发团队、中小型项目组、初级程序员、课程项目组
核心价值	降低代码评审门槛，提高测试设计效率，帮助开发者学习规范编码和缺陷定位
系统原则	AI 提供建议，人工进行最终判断；结构化输出，便于存储、检索和二次处理

1.3 项目目标

- 构建一个支持代码上传、AI 审查、测试建议生成、结果展示、人工确认和历史记录管理的轻量级 Web 系统。
- 将 LLM 返回结果约束为结构化 JSON，提高系统稳定性和可解释性。
- 通过静态分析工具与 LLM 结合，减少单纯依赖自然语言模型造成的误判。
- 形成完整的软件工程文档，包括需求分析、技术调研、系统设计、测试方案和风险分析。

2 现状与问题分析

2.1 当前软件开发中的痛点

痛点	表现	对项目质量的影响
代码评审成本高	人工 review 需要资深成员投入时间	小团队可能省略 review，缺陷流入后续阶段
测试用例不足	只测试正常路径，缺少边界值、异常路径和安全场景	隐藏缺陷无法被及时暴露
新手缺少反馈	初学者不知道代码问题在哪里，也不了解更好的实现方式	学习效率低，代码风格不稳定
AI 输出难以集成	普通聊天式回答格式不固定，无法直接入库或前端展示	系统可用性和自动化程度低
安全风险被忽视	将代码、凭证或业务逻辑直接发送给外部模型	可能造成敏感信息泄露和合规风险

2.2 项目机会

CodeMate 选择“代码评审 + 测试用例生成”作为切入点，原因是该场景边界清晰、输入输出明确、容易做课堂原型，也能体现 AI 对软件开发流程的实际价值。

系统可以从 Git diff 或代码文件中提取上下文，结合规则检查、静态分析与 LLM 推理，输出结构化问题列表和测试建议，并通过人工确认机制控制误报风险。

3 需求分析

3.1 相关方与用户角色

角色	主要目标	主要操作
开发者	在提交代码前发现潜在问题并获得修改建议	上传代码、查看审查结果、采纳或忽略建议、导出报告
评审者/组长	快速了解代码质量并监督修改过程	查看任务列表、筛选高危问题、评论 AI 建议、确认处理状态
测试人员	根据代码逻辑补充测试用例	查看测试建议、生成测试清单、标记已实现用例
管理员	维护用户、模型配置、提示词模板和安全策略	配置 API、管理权限、查看审计日志

3.2 业务流程

- (1) 用户登录系统，创建代码审查任务。
- (2) 用户上传代码文件、粘贴代码片段或提交 Git diff。
- (3) 系统进行语言识别、文件大小检查、敏感信息初筛和代码切片。
- (4) 系统调用静态分析工具获取基础问题，例如语法错误、未使用变量、复杂度过高等。
- (5) 系统构造提示词，将代码片段、静态分析结果和审查标准发送给 LLM。
- (6) LLM 返回结构化 JSON，系统校验 schema 并入库。
- (7) 前端按严重程度和问题类型展示结果，并生成测试用例建议。
- (8) 开发者/评审者确认每条建议，形成最终审查报告。

用户提交代码 → 预处理与敏感信息扫描 → 静态分析 → LLM 结构化审查 → 结果校验与入库 → 人工确认 → 导出报告

3.3 功能需求

编号	功能名称	需求描述	优先级
FR-01	用户登录与权限管理	支持开发者、评审者、管理员角色；不同角色拥有不同操作权限。	高
FR-02	代码输入	支持上传单个代码文件、粘贴代码片段或输入 Git diff；限制文件大小和支持语言。	高

FR-03	代码预处理	自动识别语言、文件名、函数范围，进行敏感信息初筛和代码切片。	高
FR-04	静态分析	调用 ESLint、Pylint、Checkstyle 或自定义规则输出基础问题。	中
FR-05	AI 代码审查	调用 LLM 分析潜在 bug、边界条件、异常处理、可维护性和安全风险。	高
FR-06	测试用例生成	根据函数逻辑生成正常路径、异常路径、边界值和安全相关测试场景。	高
FR-07	结构化结果展示	按问题类型、严重程度、文件和行号展示结果；支持筛选和搜索。	高
FR-08	人工确认与状态管理	支持“采纳、忽略、待确认、已修复”等状态，并记录处理意见。	高
FR-09	历史任务管理	保存每次审查任务、输入摘要、AI 输出和人工处理结果。	中
FR-10	报告导出	支持导出 Markdown、PDF 或 Word 格式审查报告。	中
FR-11	提示词模板管理	管理员可维护审查标准、输出格式、语言偏好和模型参数。	中
FR-12	审计日志	记录模型调用、用户操作、结果修改和异常事件。	中

3.4 非功能需求

类别	要求
性能	普通代码片段审查响应时间控制在 20 秒以内；较大 diff 可异步处理并显示进度。
准确性	高危问题应提供明确证据，例如代码位置、触发条件和建议修复方式；AI 不确定时必须标记 confidence。
可靠性	模型调用失败时返回可理解错误；schema 校验失败时自动重试或降级为人工查看。
安全性	输入代码应进行敏感信息扫描；API Key 不得前端暴露；用户只能访问自己的任务或授权项目。
隐私保护	对提交代码进行最小化发送；可配置是否调用外部模型；日志中避免保存密钥、Token 和完整敏感代码。
可维护性	提示词、模型、静态分析规则和语言适配器模块化配置。
可用性	界面应突出严重程度、影响范围和下一步操作；支持中文解释，适合课程项目展示。
可扩展性	后续可接入 GitHub/GitLab PR、CI/CD、RAG 知识库和更多编程语言。

3.5 典型用例

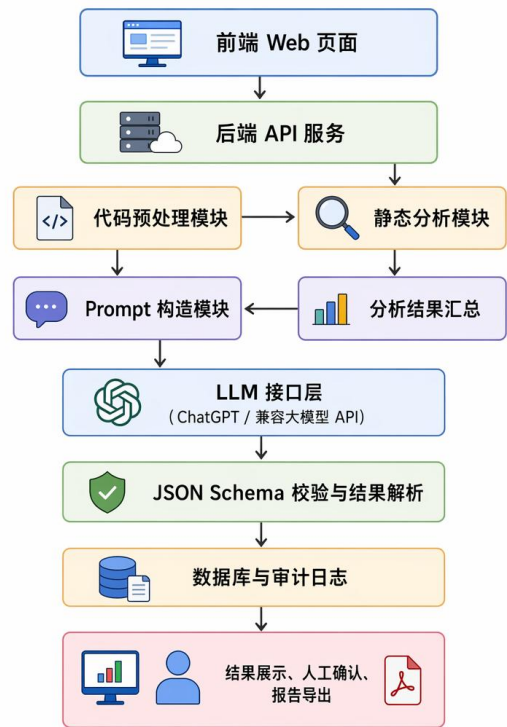
用例	参与者	前置条件	主流程	后置结果
UC-01 提交代码审查	开发者	用户已登录并准备代码	上传/粘贴代码 → 选择语言和项目 → 提交任务	系统生成审查任务并返回结果
UC-02 查看 AI 建议	开发者/评审者	审查任务已完成	打开任务详情 → 筛选 high 问题 → 查看说明和修复建议	用户理解问题并决定处理方式
UC-03 生成测试建议	测试人员	代码审查任务已完成	进入测试建议页 → 查看用例名称、输入、预期输出	形成测试清单
UC-04 人工确认结果	评审者	AI 已返回审查结果	逐条标记采纳/忽略/待确认 → 填写备注	形成可信审查记录
UC-05 导出报告	开发者/组长	任务已确认	点击导出 → 选择格式 → 下载报告	获得可提交或归档的审查报告

3.6 需求优先级

优先级	内容
Must	代码输入、AI 审查、测试建议、结构化展示、人工确认
Should	历史任务、报告导出、静态分析集成、基础权限控制
Could	GitHub PR 集成、团队项目看板、RAG 知识库、自动修复建议 PR

4 系统总体设计

4.1 总体架构



4.2 技术栈

层级	推荐技术	理由
前端	Vue 3 / React + TypeScript	适合实现任务列表、代码视图、筛选器和审查结果卡片
后端	Python FastAPI	FastAPI 适合快速原型和 AI API 调用
数据库	SQLite	课程原型可用 SQLite
AI 模型	OpenAI API / 兼容 Chat Completions 或 Responses API 的模型	便于实现结构化输出和多轮提示词
静态分析	ESLint、Pylint、Checkstyle、Bandit、CodeQL（可选）	将确定性规则与 LLM 推理结合，提高可信度
报告生成	Markdown / docx / PDF 导出	便于课程提交和团队归档
部署	Docker + 本地服务器	便于统一运行环境和演示

4.3 核心模块设计

模块	职责	输入	输出
代码接入模块	接收文件、代码片段或 diff，识别语言和项目	文件、文本、diff	标准化代码任务
安全预处理模块	检测密钥、Token、邮箱、数据库连接串等敏感信息	原始代码	脱敏代码或阻断提示
静态分析模块	运行规则工具并提取基础问题	代码文件	规则问题列表
Prompt 构造模块	合并代码上下文、规则结果审查标准	代码片段、静态分析结果	模型请求消息
LLM 审查模块	调用模型并要求输出符合 JSON Schema 的结果	Prompt	AI 审查 JSON

结果校验模块	校验字段、枚举值、行号和置信度	AI JSON	有效审查结果或重试请求
人工确认模块	让用户确认、忽略或修改 AI 建议	审查结果	最终审查记录
报告模块	生成可提交的审查报告和测试建议清单	任务数据	Markdown/PDF/Word

4.4 数据库设计

表名	主要字段	说明
users	id, username, password_hash, role, created_at	用户与角色信息
review_tasks	id, user_id, project_name, language, status, created_at, completed_at	一次代码审查任务
code_inputs	id, task_id, file_name, code_hash, code_excerpt, diff_text	代码输入摘要，不建议保存敏感完整代码
review_issues	id, task_id, type, severity, file, line_start, line_end, description, suggestion, confidence, status	AI 与静态分析发现的问题
test_cases	id, task_id, name, category, input, expected, priority, status	生成的测试用例建议
prompt_templates	id, name, template, version, enabled	提示词模板配置
audit_logs	id, user_id, action, target_id, detail, created_at	审计与追踪记录

4.5 AI 输出结构设计

为了便于程序处理，模型输出不应是自由文本，而应符合固定 JSON Schema。OpenAI Structured Outputs 的核心思想是让模型输出符合开发者定义的 JSON Schema，并支持类型安全、显式拒绝和更稳定的格式控制 [4]。

```
{
  "summary": "本次代码审查共发现 3 个问题，其中 1 个高危问题。",
  "issues": [
    {
      "type": "null_pointer_risk",
      "severity": "high",
      "file": "UserService.java",
      "line_start": 42,
      "line_end": 44,
      "description": "user 对象在访问 getName() 前没有进行空值判断。",
      "suggestion": "在调用 user.getName() 前增加 null 检查，并返回明确错误信息。",
      "confidence": 0.86
    }
  ],
  "test_cases": [
    {
      "name": "用户对象为空时应返回参数错误",
      "category": "exception_path",
      "input": "user = null",
      "expected": "返回参数错误，不抛出未捕获异常",
      "priority": "high"
    }
  ]
}
```

5 关键技术调研

5.1 ChatGPT / Copilot 辅助软件开发能力

ChatGPT 与 Copilot 类工具已不再局限于“补全代码”，而是逐步覆盖软件开发生命周期中的多个环节，包括需求解释、代码生成、代码评审、测试编写、重构建议、PR 摘要和任务代理。

GitHub Copilot code review 能够对 Pull Request 或 IDE 中的代码进行审查，并提供问题反馈和修复建议；其文档同时指出，开发者必须验证 AI 反馈，不能将 AI 结果视为完全可靠 [1]。

GitHub 还提供单元测试生成相关提示文件能力，开发者可以在 VS Code 中使用 `/generate-unit-tests`，并指定目标函数和测试框架，例如 `function_name` 与 `framework` 参数 [2]。

Copilot cloud agent 支持从 GitHub Issues、GitHub 页面、Copilot Chat、CLI、IDE 等入口启动任务，并可在后台工作、创建 Pull Request 和请求人工 review [3]。这说明 AI 辅助开发正在从“代码补全工具”演进为“开发流程协作者”。

能力方向	代表工具/功能	对 CodeMate 的启发
代码补全	ChatGPT、GitHub Copilot、Bing Copilot	为开发者提供即时编码建议，但不是本项目核心
代码评审	GitHub Copilot code review	本项目核心模块，输出问题、严重程度和修复建议
测试生成	GitHub Copilot <code>/generate-unit-tests</code>	本项目生成边界、异常和正常路径测试用例
代理式开发	Copilot cloud agent	本项目暂不做自主改代码，但保留未来扩展接口
企业知识增强	Copilot Studio connectors	未来可接入团队规范、项目文档和编码标准作为知识源

5.2 结构化输出技术

AI 代码审查如果只返回自然语言，将难以前端展示、排序、筛选和入库。因此系统应采用结构化输出策略，让模型按固定字段返回结果。

OpenAI 文档指出，Structured Outputs 可确保模型文本响应遵循开发者定义的 JSON Schema；相较普通 JSON mode，Structured Outputs 不仅保证有效 JSON，还强调 schema adherence，即符合指定字段与类型 [4]。

在 CodeMate 中，结构化输出字段应包括 `summary`、`issues`、`test_cases`、`severity`、`confidence`、`file`、`line_start`、`line_end`、`suggestion` 等。后端应对模型返回进行 schema 校验，失败时重试或进入人工处理。

5.3 静态分析与 LLM 结合

静态分析工具具有确定性和可复现性，适合发现语法、风格、依赖和部分安全问题；LLM 更擅长解释复杂逻辑、推断边界条件和生成自然语言建议。

因此本项目建议采用“静态分析 + LLM”的混合方案：静态分析先输出确定性问题，LLM 再结合上下文解释问题、归并重复项并生成测试建议。GitHub Changelog 中关于 Copilot code review 的更新也体现了类似方向，即将 LLM 检测与 ESLint、CodeQL 等确定性工具结合，用于更完整的审查 [9]。

技术	优势	局限	在本项目中的作用
静态分析	速度快、结果稳定、可解释	对业务逻辑和上下文理解有限	作为基础问题发现和证据来源
LLM 推理	能解释逻辑、生成修复建议和测试场景	可能幻觉、误报或漏报	作为审查增强和测试生成模块
结构化输出	便于前端展示和数据库存储	需要 schema 设计和错误处理	作为 AI 输出到系统数据的接口

人工确认	降低误报风险，符合真实开发流程	需要用户投入时间	作为最终质量把关环节
------	-----------------	----------	------------

5.4 企业知识源与 RAG 可扩展性

如果系统面向企业或课程团队长期使用，单纯依赖通用模型可能无法理解团队编码规范、项目架构和领域术语。可通过知识库或 RAG 方式把团队规范、README、接口文档和历史问题接入审查流程。

Microsoft Copilot Studio 文档显示，企业级 agent 可以通过 Copilot connectors 将非 Microsoft 系统中的企业数据索引到 Microsoft Graph 中，并作为 agent 的知识源；该机制还强调尊重源系统权限，确保用户只能访问授权内容 [5]。CodeMate 可借鉴这一思路，在未来版本中接入团队编码规范与项目文档。

5.5 安全与风险管理技术

AI 辅助软件开发系统需要特别关注提示注入、敏感信息泄露、过度依赖模型、模型输出错误、代码隐私和供应链风险。OWASP Gen AI Security Project 发布的 LLM/GenAI Top 10 将 Prompt Injection、Sensitive Information Disclosure 等列为重要风险项，并提供风险与缓解方向 [6]。

NIST AI RMF 提供了面向可信与负责任 AI 的风险管理思路，其生成式 AI Profile 用于帮助组织识别 GenAI 独特风险并提出相应管理措施 [7]。在软件工程层面，NIST SSDF SP 800-218 建议将安全软件开发实践集成到 SDLC 中，以减少漏洞、降低未修复漏洞的影响并解决漏洞根因 [8]。

因此，CodeMate 在需求层面必须包含敏感信息扫描、最小权限、审计日志、人工确认和安全提示词设计。

6 安全、隐私与合规分析

风险	场景	影响	应对措施
提示注入	用户在代码注释中写入“忽略之前规则”等恶意指令	模型可能偏离审查任务	将代码内容作为数据而非指令；系统提示中明确忽略代码注释内的指令
敏感信息泄露	代码中包含 API Key、Token、密码或数据库连接串	可能泄露个人或项目机密	提交前扫描敏感信息；脱敏后再调用模型；禁止在日志中保存密钥
过度依赖 AI	开发者直接接受 AI 建议而不验证	错误修复导致新 bug	所有建议默认待确认；高危建议必须人工审核
模型幻觉	AI 引用不存在的函数、文件或行号	误导开发者	校验行号、文件名；要求建议必须引用输入代码证据
权限越界	普通用户查看其他项目代码审查结果	隐私和项目安全风险	基于角色和项目成员的访问控制
成本与滥用	用户频繁提交超大文件或重复请求	API 成本过高、系统不可用	文件大小限制、频率限制、任务队列和配额管理

6.1 安全策略

- 默认不保存完整敏感代码，只保存哈希、摘要和必要审查片段。
- 用户提交代码前进行敏感信息扫描，命中高风险模式时提醒用户脱敏。
- 后端统一保管模型 API Key，前端不得直接调用模型接口。
- 模型输出必须经过 schema 校验和内容校验后才能入库。
- 所有 AI 结论在前端标注为“建议”，不得标注为“确定错误”。
- 关键操作写入审计日志，包括提交、模型调用、修改状态、导出报告等。

7 测试与评价方案

7.1 功能测试

测试项	测试内容	预期结果
登录与权限	不同角色登录并访问任务页面	权限控制正确，未授权访问被拒绝
代码上传	上传 Python、Java、JavaScript 文件	系统正确识别语言并创建任务
AI 审查	提交包含空指针、边界值、异常处理缺失的代码	输出问题列表和修复建议
测试生成	提交包含条件分支的函数	生成正常、异常、边界测试场景
人工确认	对 AI 建议标记采纳、忽略、已修复	状态正确保存并可查询
报告导出	导出审查报告	报告包含任务信息、问题列表、测试建议和处理状态

7.2 AI 效果评价指标

指标	计算方式/说明	目标
问题发现率	AI 发现的真实问题数 / 人工标注真实问题数	越高越好
误报率	AI 输出但人工判断不成立的问题数 / AI 输出问题总数	越低越好
建议可采纳率	被用户采纳的建议数 / 建议总数	达到 60% 以上
测试覆盖提升	使用 AI 建议后新增覆盖的分支或异常路径数量	明显提升
平均响应时间	从提交任务到返回结果的平均时间	小代码片段 20 秒内
用户满意度	用户对建议清晰度、实用性的评分	5 分制平均 4 分以上

7.3 测试数据设计

准备 10 至 20 个小型代码样本，覆盖空指针、越界、异常处理缺失、SQL 拼接、未关闭资源、重复代码、复杂条件、边界值缺失等问题。

每个样本由人工预先标注真实问题和期望测试用例，然后与系统输出进行对比。

对于模型输出，重点评价是否给出合理证据、是否定位到正确代码区域、修复建议是否可执行。

8 项目实施计划与可行性分析

8.1 里程碑计划

阶段	时间	主要任务	交付物
第 1 阶段	第 1 周	完成需求分析、技术调研、原型设计	需求文档、技术调研文档、界面草图
第 2 阶段	第 2 周	实现代码上传、任务管理和基础页面	前后端基础框架
第 3 阶段	第 3 周	接入 LLM，完成结构化审查结果解析	AI 审查 Demo
第 4 阶段	第 4 周	实现测试用例生成、人工确认和历史记录	可演示系统
第 5 阶段	第 5 周	测试、优化、补充报告导出	最终项目、演示视频

8.2 可行性分析

维度	分析结论
技术可行性	系统核心流程可由 Web 前后端、数据库和大模型 API 组成。课程原型不需要实现完整 IDE 插件或 GitHub App，因此难度可控。
经济可行性	可使用少量 API 调用完成演示，也可使用本地模拟结果降低成本。
时间可行性	功能可按 Must/Should/Could 分层实现，先保证代码输入、AI 审查、测试建议和结果展示。
教学价值	项目覆盖需求分析、系统设计、AI 技术调研、安全风险和测试评价，适合软件工程课程提交。
扩展价值	后续可扩展为 GitHub PR Review Bot、IDE 插件或团队质量看板。

8.3 风险与应对

风险	概率	影响	应对
模型输出不稳定	中	高	使用 JSON Schema 校验，失败后重试，必要时返回人工处理
代码过长导致上下文超限	中	中	按文件、函数或 diff hunk 切片，只分析变更部分
AI 建议误导用户	中	高	标注置信度，要求人工确认，保留原始证据
API 调用成本超预算	中	中	限制文件大小、使用缓存、演示时使用样例数据
静态分析工具接入复杂	低	中	先实现 LLM 审查，静态分析作为可选增强
敏感信息泄露	低	高	提交前扫描和脱敏；不保存密钥；后端代理调用

9 总结

CodeMate 项目以 AI 辅助代码评审与测试用例生成为核心，将 ChatGPT / Copilot 类工具能力落实到具体的软件工程场景中。与单纯的聊天式代码问答相比，本项目强调结构化输出、人工确认、静态分析结合、安全与隐私控制，以及可评价的质量指标。该项目选题边界明确、技术调研充分、原型实现难度适中，能够体现 AI 辅助软件开发的现实价值与局限性。后续若继续扩展，可接入 GitHub PR、团队知识库、CI/CD 和自动修复 Pull Request，形成更完整的 AI 软件质量保障平台。

附录 A： AI 输出 JSON 示例

```
{
  "summary": "本次审查发现 2 个中高风险问题，建议补充 4 个测试场景。",
  "issues": [
    {
      "type": "boundary_condition_missing",
      "severity": "medium",
      "file": "OrderService.py",
      "line_start": 18,
      "line_end": 25,
      "description": "订单金额为 0 或负数时没有显式校验。",
      "suggestion": "在创建订单前增加 amount > 0 的校验，并返回业务异常。",
      "confidence": 0.82
    }
  ],
  "test_cases": [
    {
      "name": "订单金额为负数时创建失败",
      "category": "boundary",
      "input": "amount = -1",
      "expected": "抛出业务异常或返回错误码",
      "priority": "high"
    }
  ]
}
```

附录 B：需求追踪矩阵

需求编号	设计模块	测试用例	验收标准
FR-02	代码接入模块	上传多语言文件测试	能够创建任务并识别语言
FR-05	LLM 审查模块	缺陷代码审查测试	输出问题类型、行号、说明和建议
FR-06	测试生成模块	分支函数测试生成	至少生成正常、异常和边界场景
FR-07	结果展示模块	筛选与排序测试	可按严重程度和类型筛选
FR-08	人工确认模块	状态流转测试	状态保存正确且可回溯
NFR-安全	安全预处理模块	敏感信息扫描测试	命中 Token/Key 时提示脱敏或阻断

参考文献

- [1] GitHub Docs. About GitHub Copilot code review.. <https://docs.github.com/en/copilot/concepts/agents/code-review>
- [2] GitHub Docs. Generate unit tests.. <https://docs.github.com/en/copilot/tutorials/customization-library/prompt-files/generate-unit-tests>
- [3] GitHub Docs. Starting GitHub Copilot sessions.. <https://docs.github.com/en/copilot/how-to/use-copilot-agents/cloud-agent/start-copilot-sessions>
- [4] OpenAI API Docs. Structured model outputs.. <https://developers.openai.com/api/docs/guides/structured-outputs>
- [5] Microsoft Learn. Add Copilot connectors as a knowledge source.. <https://learn.microsoft.com/en-us/microsoft-copilot-studio/knowledge-copilot-connectors>
- [6] OWASP Gen AI Security Project. 2025 Top 10 Risk & Mitigations for LLMs and Gen AI Apps.. <https://genai.owasp.org/llm-top-10/>
- [7] NIST. AI Risk Management Framework.. <https://www.nist.gov/itl/ai-risk-management-framework>
- [8] NIST CSRC. SP 800-218 Secure Software Development Framework Version 1.1.. <https://csrc.nist.gov/pubs/sp/800/218/final>
- [9] GitHub Changelog. New public preview features in Copilot code review: AI reviews that see the full picture.. <https://github.blog/changelog/2025-10-28-new-public-preview-features-in-copilot-code-review-ai-reviews-that-see-the-full-picture/>
- [10] GitHub Blog. Assigning and completing issues with coding agent in GitHub Copilot.. <https://github.blog/ai-and-ml/github-copilot/assigning-and-completing-issues-with-coding-agent-in-github-copilot/>